GENERATEVIDEOTOOL · MVP

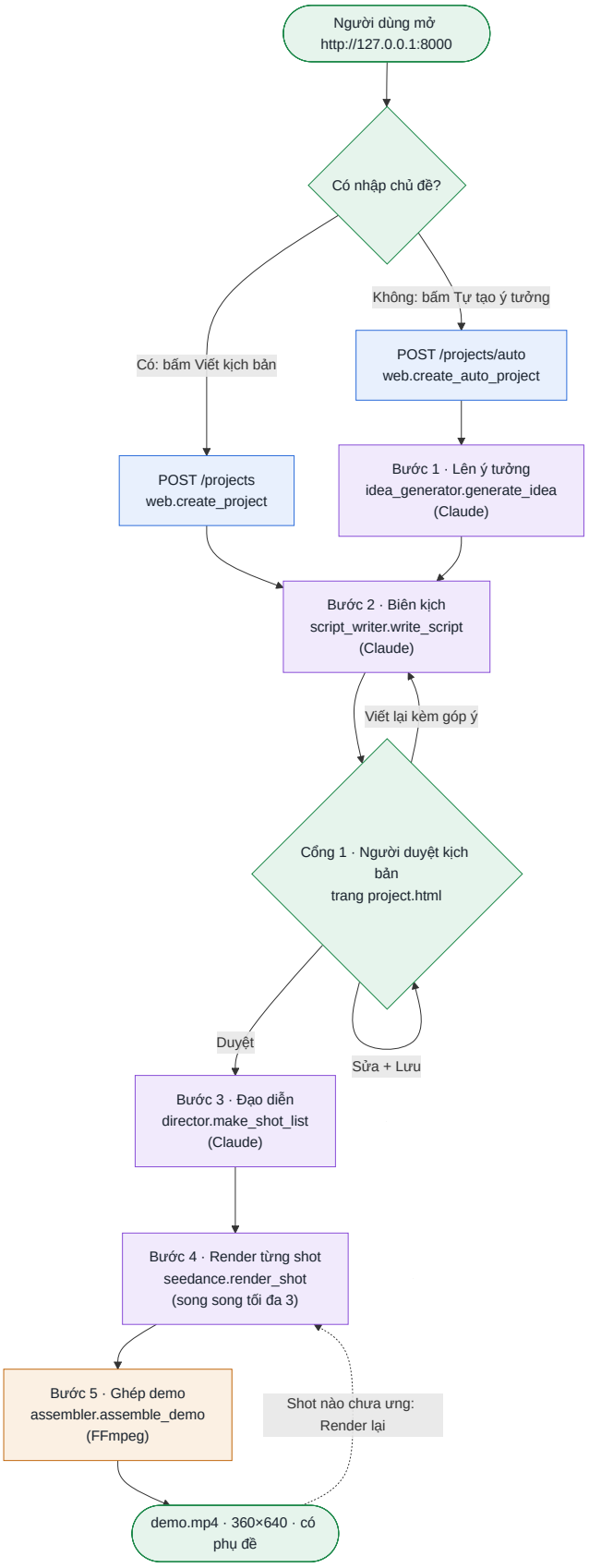
Code chạy như thế nào

Từ lúc bạn bấm nút trên web đến khi có file `demo.mp4`: ai làm gì, file nào chạy, dữ liệu đi đâu.

1. Bức tranh tổng thể

Tool là một **dây chuyền** gồm 5 trạm. Mỗi trạm nhận kết quả của trạm trước, làm xong thì ghi vào `project.json` rồi chuyển tiếp. Chỉ có một chỗ dừng lại chờ người: **Cổng 1** (duyệt kịch bản).

Người dùng AI (Claude / Seedance) Công cụ (FFmpeg) Web / điều phối



Mọi mũi tên sau trình duyệt đều chạy trong file `app/pipeline.py`, nó là "quản đốc" gọi lần lượt các trạm.

2. Chi tiết từng bước

0 Khởi động — Hệ thống

`run.py` → `app/config.py` → `app/web.py`

- `python run.py` bật web server (uvicorn) ở cổng 8000, nạp ứng dụng FastAPI trong `web.py`.
- `config.py` đọc file `.env`: API key, tên model, độ phân giải, số render song song, đường dẫn FFmpeg...
- Quyết định **chế độ thử**: nếu `MOCK_CLAUDE=auto` và `ANTHROPIC_API_KEY` trống thì Claude giả; tương tự `ARK_API_KEY` trống thì Seedance giả. Hai thứ này độc lập nhau.
- Tạo thư mục `data/projects/` nếu chưa có.

1 Bạn tạo dự án — Người dùng

`templates/index.html` → `web.create_project` hoặc `web.create_auto_project` → `pipeline.start_project` / `start_auto_project`

- Form gửi **Brief**: chủ đề, nền tảng, thời lượng, tone, đối tượng, ngôn ngữ (định nghĩa ở `schemas.Brief`).
- `storage.create` tạo mã dự án dạng `20261103-154906-b16a`, ghi `project.json` với trạng thái `writing`.
- Việc gọi AI được đẩy sang **luồng nền** (hàm `_bg`), còn trình duyệt được chuyển ngay sang trang dự án. Nhờ vậy web không bị treo trong lúc chờ AI.

2 Lên ý tưởng (chỉ khi bấm "Tự tạo ý tưởng") — Claude

`app/idea_generator.py` · hàm `generate_idea`

- Lấy danh sách ~30 chủ đề đã làm gần đây để bảo Claude **tránh trùng**.
- Claude trả về đối tượng `Idea`: chủ đề, đối tượng, tone, lý do. Hệ thống ghi đè vào Brief rồi đi tiếp sang bước 3.
- Chế độ thử: chọn ngẫu nhiên 1 trong 8 ý tưởng mẫu.

3 Viết kịch bản Claude · Biên kịch

`app/script_writer.py` · hàm `write_script` → `claude_client.ask_structured`

- Gửi Brief cùng "luật biên kịch": hook 3 giây đầu, mỗi cảnh 2–5 giây, tổng đúng thời lượng, cảnh phải vẽ được bằng AI, có CTA cuối.
- **Ép đúng định dạng**: `ask_structured` buộc Claude trả kết quả qua một "tool" có schema JSON của `Script`. Sai schema thì thử lại 1 lần.
- Kết quả `Script` gồm: tiêu đề, hook, danh sách cảnh (`visual` = mô tả hình, `voiceover` = câu phụ đề, `duration`), CTA, gợi ý nhạc.
- Lưu vào `project.json`, chuyển trạng thái sang `review_script`.

4 Cổng 1 · Duyệt kịch bản Người dùng

`templates/project.html` → `web.save_script`

- Bạn sửa trực tiếp tiêu đề, hook, CTA, từng cảnh (thời lượng, hình ảnh, lời thoại), hoặc xoá cảnh. Cảnh được đánh số lại sau khi xoá.
- Ba nút: **Lưu** (chỉ ghi lại), **Viết lại** (gửi bản cũ kèm góp ý cho Claude, quay lại bước 3), **Duyệt** (sang bước 5).
- Đây là chỗ duy nhất tool dừng lại chờ bạn, trước khi tốn tiền render.

5 Phân cảnh Claude · Đạo diễn

`app/director.py` · hàm `make_shot_list`

- Biến mỗi cảnh thành 1 **shot** có ngôn ngữ điện ảnh: cỡ cảnh, góc máy, chuyển động máy, ánh sáng, diễn xuất.
- Viết **prompt tiếng Anh** 40–90 từ cho Seedance. Mô tả nhân vật cố định (`characters`) được lặp lại trong mọi prompt để nhân vật giống nhau giữa các shot.
- Kiểm tra: số shot phải khớp số cảnh, thiếu cảnh nào thì báo lỗi.
- Mỗi shot được gán một **seed** ngẫu nhiên, trạng thái chuyển sang `rendering`.

6 Render video từng shot — Seedance

`pipeline._render_all` → `_render_one` → `seedance.render_shot`

- Chạy **song song** tối đa `MAX_PARALLEL_RENDERS` shot (mặc định 3).
- Thời lượng gửi lên được làm tròn lên và kẹp trong 4–15 giây (`api_duration`). Cảnh 3 giây vẫn render 4 giây, phần thừa cắt ở bước ghép.
- Quy trình mỗi shot: **tạo task** (POST) → **hỏi trạng thái** 8 giây/lần, tối đa 20 phút → **tải mp4** về `shots/shot_01_<seed>.mp4`.
- Lỗi 429 (quá tải) hoặc 5xx: tự thử lại sau 3, 6, 12, 24 giây. Lỗi 4xx khác (sai key, sai model) thì dừng ngay.
- Chế độ thử: FFmpeg tạo màn hình sọc màu (`testsrc2`). Đây chính là hình bạn thấy trước đó.

> Nếu có shot lỗi, các shot khác vẫn chạy xong. Dự án chuyển sang `failed` và báo shot nào lỗi để bạn bấm "Render lại".

7 Ghép thành demo — FFmpeg

`app/assembler.py` · hàm `assemble_demo`

- **Chuẩn hoá** từng clip: phóng/cắt về 360×640, 24 fps, cắt đúng thời lượng cảnh. Clip nào ngắn hơn thì kéo dài khung cuối.
- **Nối** các clip theo thứ tự cảnh thành `joined.mp4`.
- **Phụ đề**: tạo file `subs.srt` từ `voiceover` của từng cảnh, rồi "đốt" chữ vào hình. Nếu FFmpeg không hỗ trợ phụ đề thì xuất bản không có chữ.
- Xuất `demo.mp4`, trạng thái sang `demo_ready`. Trang dự án hiện video.

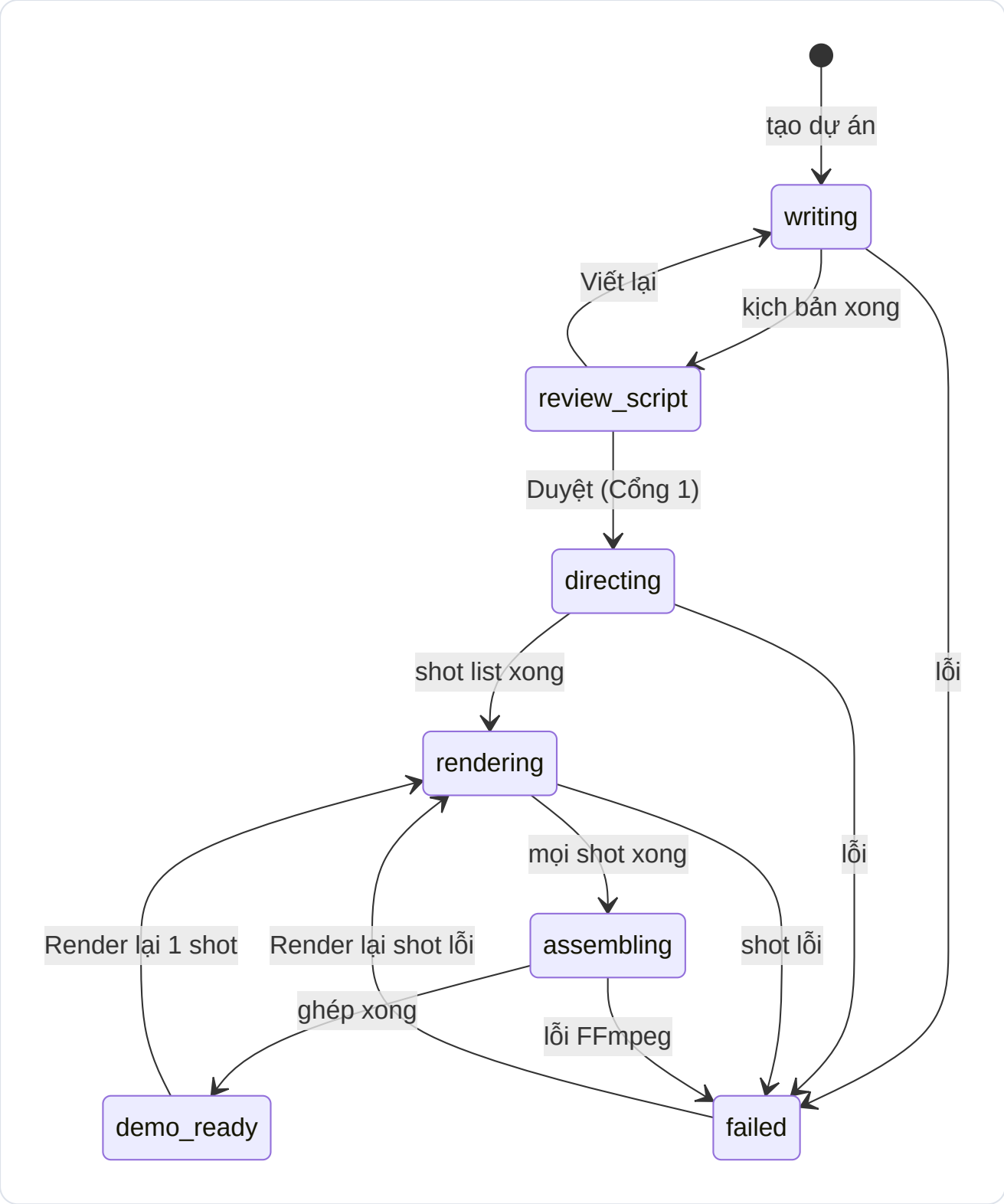
8 Render lại 1 shot (tuỳ chọn) — Người dùng

`web.rerender` → `pipeline.rerender_shot`

- Đổi seed mới (và prompt mới nếu bạn sửa), chỉ render lại đúng shot đó, rồi tự ghép lại demo. Các shot khác giữ nguyên, không tốn thêm tiền.

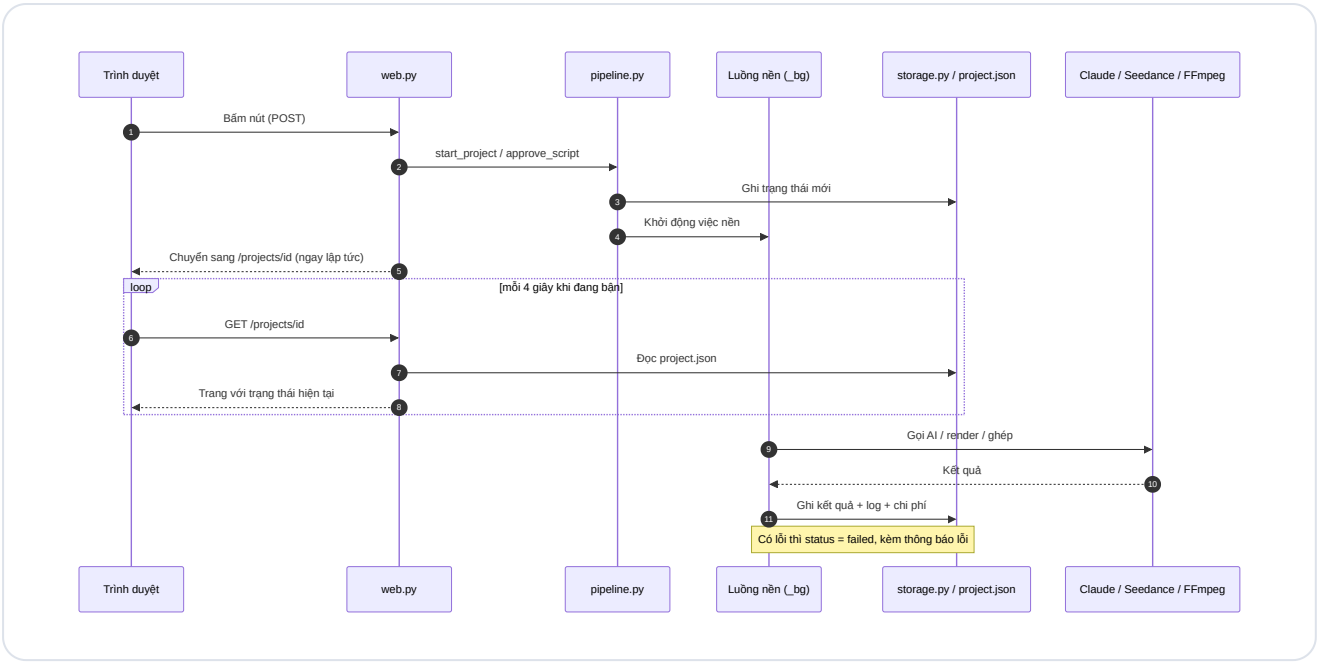
3. Trạng thái của một dự án

Trường `status` trong `project.json` cho biết dự án đang ở đâu. Khi đang ở trạng thái "bận" (4 trạng thái màu tím), trang tự làm mới mỗi 4 giây.



4. Vì sao web không bị treo khi AI đang chạy

Mỗi thao tác dài (gọi Claude, render, ghép) chạy trong một **luồng nền**. Trình duyệt không đợi kết quả mà chỉ đọc lại `project.json` mỗi 4 giây.



- `storage.update` dùng khoá (`RLock`) để nhiều luồng render song song không ghi đè lên nhau.
- File được ghi ra `.tmp` rồi mới đổi tên, nên `project.json` không bao giờ bị ghi dở.
- Mỗi lần gọi AI đều ghi vào `cost_log` (token Claude, usage Seedance), để sau này tính giá thành mỗi video.

5. Chế độ thử và chế độ thật khác nhau ở đâu

BƯỚC	CHẾ ĐỘ THỬ (CHƯA CÓ KEY)	CHẾ ĐỘ THẬT
Lên ý tưởng	Chọn ngẫu nhiên 1 trong 8 ý tưởng mẫu	Claude nghĩ ý tưởng mới, tránh trùng
Kịch bản	Luôn là 4 cảnh mẫu "dậy muộn", chèn chủ đề vào cảnh 2	Claude viết theo đúng Brief
Phân cảnh	Ghép mô tả cảnh thành prompt đơn giản	Claude đạo diễn từng shot
Render	FFmpeg tạo màn hình sọc màu	Seedance tạo video có nhân vật, hoạt cảnh
Ghép demo	Giống nhau: FFmpeg chạy thật trong cả hai chế độ	

Key Claude và key Seedance độc lập. Bạn có thể dùng Seedance thật với kịch bản mẫu, hoặc ngược lại.

6. Bản đồ file

FILE	VAI TRÒ	HÀM CHÍNH
`run.py`	Điểm khởi động	bật uvicorn
`config.py`	Đọc `.env`, quyết định chế độ thử	`use_mock_claude`, `use_mock_seedance`
`schemas.py`	"Khuôn" dữ liệu: Brief, Idea, Script, ShotList, Project, Status	các class Pydantic
`web.py`	Nhận thao tác từ trình duyệt	`create_project`, `create_auto_project`, `save_script`, `rerender`
`pipeline.py`	Quản đốc: gọi các trạm theo thứ tự, chạy nền	`start_project`, `approve_script`, `_render_all`, `_assemble`
`claude_client.py`	Gọi Claude, ép trả đúng schema	`ask_structured`
`idea_generator.py`	Agent lên ý tưởng	`generate_idea`
`script_writer.py`	Agent biên kịch	`write_script`
`director.py`	Agent đạo diễn	`make_shot_list`
`seedance.py`	Gọi API Seedance, chờ, tải video	`render_shot`, `SeedanceClient`
`assembler.py`	FFmpeg: chuẩn hoá, nối, phụ đề	`assemble_demo`
`storage.py`	Đọc/ghi `project.json` an toàn	`create`, `load`, `update`, `log`

7. Dữ liệu một dự án nằm ở đâu

```
data/projects/20261103-154906-b16a/
├─ project.json      # toàn bộ trạng
├─ shots/
│  ├─ shot_01_<seed>.mp4  # video từn
│  └─ shot_02_<seed>.mp4
├─ tmp/              # file trung gia
│  ├─ norm_01.mp4 …
│  ├─ joined.mp4
│  └─ subs.srt
└─ demo.mp4          # kết quả cuối
```

Bên trong project.json

- `brief` : yêu cầu ban đầu (hoặc ý tưởng AI tạo)
- `status` : đang ở bước nào
- `script` : kịch bản sau Cổng 1
- `shot_list` : shot + prompt Seedance
- `renders` : seed, task_id, file, lỗi của từng shot
- `log` : nhật ký theo giờ
- `cost_log` : chi phí từng bước